Introduction to Software Project Management

Unit_1

Introduction

- “Most software projects fail due to mismanagement, not just bad code. This course will teach you how to plan, manage, and succeed in real-world software development.”
- “More than 60% of software projects run over time or budget. Why? Let’s find out.”

Software Project Management (SPM)

- SPM is the discipline of planning, organizing, and managing software projects.
- Focuses on delivering software products on time, within budget, and meeting quality.
- Involves resource management, risk management, cost estimation, and change control.
- Ensures alignment between technical execution and business goals.

Historical Background and Context

- 1960s: Software crisis revealed project failures in cost, schedule, quality.
- 1970s–80s: Waterfall model introduced for structure; rigid sequential phases.
- 1990s: PMBOK and PRINCE2 formalized project management knowledge areas.
- 2000s–present: Agile and iterative methods introduced to improve flexibility and user involvement.

Software Crisis?

- The **“software crisis”** refers to a period during the **late 1960s through the 1970s** when large-scale software projects increasingly **failed to meet expectations** — they were **over budget, behind schedule, and often didn’t work**.
- This crisis led to a turning point in how software development was perceived and managed.

Why Did the Crisis Occur?

- **Exploding Demand for Software**
  - The rise of mainframe computers in government, military, and business meant software was needed on **a much larger scale**.
  - Projects became **too complex** to be handled by informal or ad-hoc coding.
- **Poor Project Planning and Estimation**
  - Projects lacked structured management.
  - Delivery deadlines and budgets were often **wildly inaccurate**.
- **Changing or Unclear Requirements**
  - Clients often didn't know exactly what they wanted.
  - Developers built the wrong product — or the right product too late.
- **No Formal Processes**
  - There were **no standardized methods** for testing, requirements gathering, or version control.
  - Coding was seen as a "craft" rather than an engineering discipline.
- **Lack of Tools and Team Coordination**
  - No IDEs, no Git, no Agile — developers often worked in isolation.
  - Communication breakdowns were common in large teams.

Notable Project Failures

- **IBM OS/360**
  - One of the earliest and largest software projects. It was years late and over budget by hundreds of millions of dollars. But it also sparked many innovations (e.g., modular programming, testing practices).
- **US Air Defense System (SAGE)**
  - Though pioneering, it highlighted the difficulty of managing **real-time, mission-critical systems** with thousands of lines of code.

Resulting Reactions

- **Birth of Software Engineering (1968)**
  - The term *"software engineering"* was coined at a NATO conference in Germany.
  - The goal: apply engineering discipline to software development.
- **Introduction of Models and Frameworks**
  - Waterfall Model (Winston Royce, 1970): introduced phased development.
  - Structured programming and modularization began to emerge.
- **Emergence of Software Project Management**
  - Project management became **a distinct discipline** in software development.
  - Tools like **PERT charts, Gantt charts**, and estimation techniques were adapted from civil engineering.

Impact on Modern SPM

- The crisis revealed that **coding alone is not enough** — software needs to be planned, budgeted, tested, and reviewed like any other product.
- It pushed the industry toward:
  - **Standard methodologies** (PMBOK, PRINCE2)
  - **Disciplined testing and versioning**
  - **Project management roles** and training
  - **Modern practices** like Agile and DevOps to respond to change more rapidly

Need and Importance of SPM

- Software development is complex, evolving, and often unpredictable.
- SPM provides structure, planning, and visibility into project status.
- Minimizes risks through early identification and mitigation strategies.
- Ensures stakeholder alignment, user satisfaction, and product quality.

Advantages of Software Project Management

- Improved planning, scheduling, and resource allocation.
- Better communication among stakeholders and team members.
- Enhanced risk management and quality assurance processes.
- Greater transparency, traceability, and project control.

Disadvantages and Challenges

- May introduce overhead with excessive documentation and process.
- Rigid management may reduce agility and innovation.
- Requires skilled managers and leadership commitment.
- Not a guarantee of success—relies on proper execution and team collaboration.

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Product Nature	Tangible, visible physical structure	Intangible, code-based software
Design Finalization	Must be finalized before construction begins	Can be revised during development (flexible scope)
Change Flexibility	Changes after construction starts are costly	Can implement changes even late in development
Stakeholder Involvement	Involved mostly in early and final stages	Involved throughout (sprints, feedback loops, testing)

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Testing & Quality Check	Inspections done at milestones (foundation, wiring)	Continuous testing (unit, integration, UAT, beta testing)
Progress Visibility	Visibly trackable (walls, windows, paint, etc.)	Invisible progress (tracked via features, tests, demos)
Dependencies	Materials, labor, weather, land, regulations	APIs, platforms, users, device compatibility
Methodology	Waterfall (sequential: plan → build → deliver)	Agile/Iterative (develop, test, review, repeat)

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Methodology	Waterfall (sequential: plan → build → deliver)	Agile/Iterative (develop, test, review, repeat)
Failure Impact	Failure is often visible and costly to reverse	Failures may be silent (bugs, performance issues)
Resource Types	Bricks, cement, plumbers, electricians	Developers, designers, testers, cloud platforms
Documentation Importance	Blueprints essential and legally binding	Documentation optional or lightweight in Agile
Post-Delivery Changes	Major renovations needed (costly, time-consuming)	Can push app updates anytime (via app store or servers)

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Nature of Product	Tangible, physical product (building, device). Progress is visible as concrete stages completed.	Intangible product (software code and functionality). Progress is less visible, often tracked via working features or milestones.
Requirements Stability	Typically more stable after initial planning; changes are difficult and costly once execution starts.	Requirements are prone to change; project often accommodates evolving needs and refinements throughout its life cycle. Agile processes embrace change.

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Flexibility to Change	Low flexibility – constrained by physical laws and irreversible decisions (can't easily "un-build" part of a bridge). Revisions can cause major delays/costs.	High flexibility – design is malleable. Can often revise or pivot the software even late in development at relatively lower cost. Iteration and course-correction are common.
Resource Focus	Mix of human resources and physical resources (materials, equipment). Management includes supply chain and logistics for physical items.	Primarily human resources (developers, designers, testers). Management focuses on team skills, communication, and productivity. The work is people-intensive with minimal physical material concerns.

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Approach & Lifecycle	Often linear and milestone-based (e.g., design → build → deliver). Uses traditional PM tools (Gantt charts, critical path) with well-defined phases.	Often iterative/incremental (e.g., multiple development sprints). Uses both traditional PM tools and software-specific frameworks (Agile/Scrum boards, continuous integration). Development and management activities may overlap in cycles.
Quality Assurance	Quality checks at specific milestones (inspections, final testing of product functionality or safety). Defects often found late are hard to fix without rework.	Continuous quality assurance (ongoing testing, code reviews throughout development). Bugs can be fixed iteratively. Software quality includes metrics like reliability, performance

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Stakeholder Involvement	Stakeholders (clients, sponsors) involved mainly at start (requirements/design approval) and end (acceptance), with periodic status reports. External parties (suppliers, regulators) play roles at specific points.	Stakeholders (clients, end-users) ideally engaged throughout. Frequent demos, feedback cycles, and updates are provided to ensure the software meets expectations. Fewer external dependencies beyond the client, enabling focus on the customer-team feedback loop.

Brainstorming:

- “Why do you think traditional project management doesn’t work well for software?”

Key Roles in Software Project Management

- Project Sponsor: Provides funding and strategic direction.
- Project Manager: Oversees planning, execution, and control.
- Product Owner/BA: Defines requirements and prioritizes features.
- Software Developers: Build and maintain software functionality.

Additional Roles in SPM

- QA/Testers: Ensure software quality through testing strategies.
- UI/UX Designers: Improve usability and user satisfaction.
- Scrum Master: Facilitates Agile ceremonies and removes blockers.
- Stakeholders: Provide feedback, validate deliverables, and accept final product.

Activity idea:

- Form groups and assign each group a role. Ask them to describe what that role contributes to a software project.

Success Example: Nagarik App (Government of Nepal)

- **Project Description:** Nagarik App is a digital platform launched by the Government of Nepal to provide citizens with access to various public services (citizenship details, PAN registration, educational certificates, etc.) through a single mobile app.
- **Relevance to SPM:**
- **Clear goals and vision** defined by the Ministry of Communication and Information Technology.
- Project followed an **iterative development model**, launching in phases with increasing features.
- Involved **multiple stakeholders**: NITC, private software firms, NEA, MoHA, etc.
- **Challenges managed**: user data integration, privacy concerns, and testing on various devices.
- **SPM Takeaways:**
- Strong coordination among government bodies and vendors.
- Scope was phased to ensure initial delivery and later enhancements.
- Regular public feedback was incorporated—showing **Agile principles** in public sector.

Failure/Risk Example: National ID Management Project

- **Project Description:** The National ID card initiative aimed to implement biometric-based identification for all citizens. However, the rollout has been **delayed by several years**.
- **Relevance to SPM:**
- Poor **stakeholder coordination** (MoHA, Department of National ID, local governments).
- Lack of **technical capacity planning** and underestimated integration issues.
- Issues with **procurement delays**, vendor management, and infrastructure.
- **SPM Learnings:**
- Weak risk management and over-ambitious timelines led to missed deadlines.
- **Inadequate communication** between software team and field-level implementers.
- Project scope and requirement analysis were not clear during planning.

Private Sector Success: eSewa Mobile Wallet

- **Project Description:** eSewa, Nepal's first digital wallet, successfully scaled into a national payments platform offering utility payments, remittance, airline booking, and more.
- **SPM Highlights:**
- **Incremental feature release**: launched core wallet first, then added services.
- **User-centric design** led by continuous testing and feedback.
- Strong **project leadership** and resource allocation helped it scale reliably.
- **SPM Application:**
- Used Agile-style iterative development.
- Prioritized **security, performance, and scalability** as key project goals.
- **Stakeholder alignment** (banks, NRB guidelines, users) was consistently maintained.

Online Driving License Application System (DoTM)

- **Context:**
  The Department of Transport Management (DoTM) launched an online license application portal to digitize the process of applying for a driving license.
- **Challenges & SPM Failures:**
- **System crashes frequently** during peak times due to poor load testing.
- **No user feedback loop** — many users struggled with verification and appointment booking.
- **Inadequate stakeholder coordination** with local offices and exam centers.
- **Lack of scalability planning**: the system wasn't prepared for the nationwide load.
- **SPM Lessons:**
- Absence of performance testing and risk mitigation plans.
- No clear communication plan or user support strategy.
- Failure to anticipate user volume growth.

Online Education Portals During COVID-19 (Various Municipalities)

- **Context:**
  During the pandemic, many municipalities launched learning portals for public school students.
- **Challenges & SPM Failures:**
- Many platforms **were not mobile-friendly** — majority of target users accessed via phones.
- **Lack of clear requirements and user research** — no pilot testing before full deployment.
- **Content management was poor**, and most portals were inactive after a few weeks.
- **Low adoption** by teachers and students due to poor usability and lack of training.
- **SPM Lessons:**
- Ignoring end-user needs in requirement analysis leads to low usability.
- Failure to plan for user training and support caused abandonment.
- No stakeholder engagement (e.g., teachers, parents) in the development process.

National eHealth Project (Ministry of Health)

- **Context:**
  A national plan to digitize hospitals and health posts, including EMR (Electronic Medical Records) and patient tracking systems.
- **Challenges & SPM Failures:**
- **Delayed procurement and unclear technical specifications.**
- **Fragmented implementation** — different systems were installed in different hospitals without integration.
- **Vendor mismanagement** and absence of a unified roadmap caused misalignment.
- **Lack of training** and IT support at local levels meant systems weren't used even after delivery.
- **SPM Lessons:**
- No unified requirement specification or architecture plan.
- Poor coordination across central and provincial levels.
- Insufficient planning for capacity building and sustainability.

Election Commission's Real-Time Vote Counting System (Attempted 2017)

- **Context:**
  ECN attempted to implement real-time vote counting dashboards.
- **Challenges & SPM Failures:**
- **System failed under load** during peak result hours.
- Poor **data synchronization** between local vote centers and central servers.
- **Inconsistent user interfaces** led to miscommunication in early results.
- Project not thoroughly tested before real deployment.
- **SPM Lessons:**
- Lack of stress testing and fallback mechanisms.
- Inadequate disaster recovery and data integrity checks.
- Last-minute deployment without stakeholder training.